QueryMend

A talk-to-your-database agent that notices when it is wrong, fixes what it can, and says so when it can't.

Agentic AI, PBA 1 and PBA 2. Topic 17: an agent that detects and recovers from its own tool failures.
Group 17, AIA_05, MIT School of Computing, Pune. Swaraj Mandre (ADT23SOCB1192), Saumya Joshi (ADT23SOCB0997), Siddhant Pawar (ADT23SOCB1117).
github.com/Swaraj-Mandre/FailureDetectionAgent. Runs fully offline: qwen2.5:3b and nomic-embed-text through Ollama.

~~Rs 24,03,00,20~~ Our first agent reported this as September's revenue, with full confidence. The real figure is **Rs 2,40,300**. The tool had read prices in paise. Nothing crashed, so nothing noticed. This project is about catching that.

4 → 0
wrong answers given to the user, out of 15 runs (PBA 1 vs PBA 2)

0 → 3
silent corruptions caught, out of 3

10 → 0
runs where a bad number spread into the next answer, Monitor off vs on

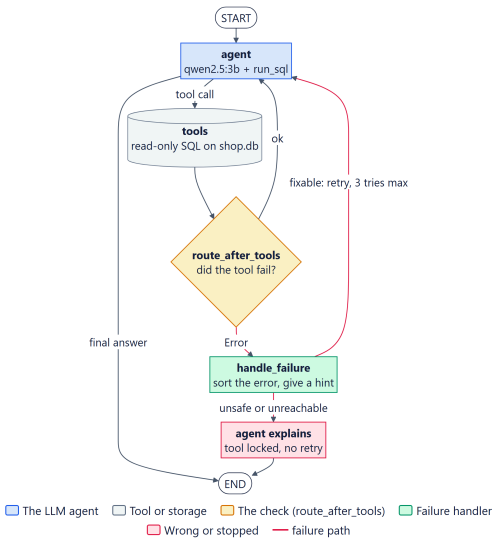
2 to 3x
model calls when something needs fixing. The price of checking.

1 The problem

A text-to-SQL agent can fail in three ways. **Loud:** the tool throws an error. **Quiet:** the SQL runs and the number is wrong. **Spreading:** the next turn builds on the wrong number. Our shop database (Sahyadri Stores: 40 customers, 25 products, 300 orders, April to September 2026) has no "amount" column on purpose, so revenue must be worked out, and the model can get it wrong. PBA 1 catches the loud failures. PBA 2 catches all three.

2 PBA 1: one agent that notices its tool failing

Units 1 and 2



One tool, `run_sql`: read-only, one SELECT at a time, and a clear `Error:` message on bad input (write words, several statements, unknown columns). `route_after_tools` checks every tool result of the turn and sends failures to `handle_failure`.

What the slide version did

Sent one generic message for every failure. The agent retried a dead database, sent the same broken query 7 times, and once told a user to run a DELETE by hand.

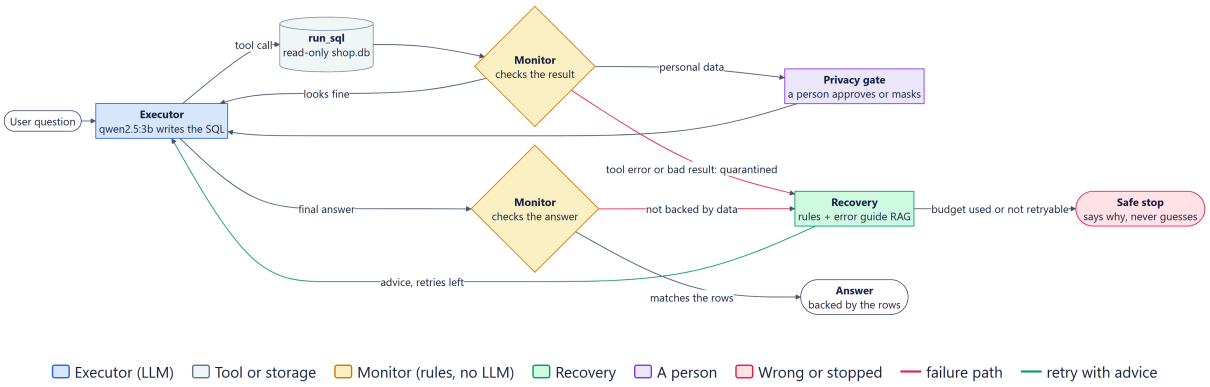
Our refinement

Classify the error, give a specific hint, cap retries at 3, detect repeated queries, and lock the tool when retrying cannot help (unsafe or unreachable). Details: `ta1/REFINEMENT_NOTES.md`.

Blind spot

Nothing looks at the data, so a wrong number with no error goes straight to the user. That is why PBA 2 has a Monitor.

3 PBA 2: Executor, Monitor and Recovery Units 3 and 4



One LangGraph with one shared message history. Red paths are failures, green is Recovery's retry. The retry budget (3 per question) is the hop limit. Colours match the roles below.

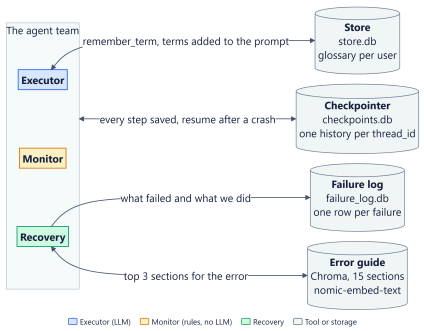
4 Roles U4-1

	Executor	Monitor	Recovery	Privacy gate (a person)
Goal	Turn the question into SQL, run it, answer from the rows	Nothing failed, impossible or unapproved reaches another agent or the user	After a failure, a corrected retry or a safe stop	Decide if personal data is shown
Tools	`run_sql`, `remember_term`, qwen2.5:3b	None. Python rules over the rows, the SQL and the shop's real limits	Error guide RAG (Chroma), failure log	LangGraph `interrupt`
Inputs	Conversation, glossary, advice	Each SQL result with its query, each final answer	Monitor's verdict, failed SQL, budget left	Columns that look personal (email, phone)
Outputs	A tool call or a final answer	ok, failure (with a type) or needs approval. Swaps a bad result for a quarantine note	Typed advice `[recovery -> executor]` or a safe stop; a log row	Approve, or reject with columns masked
Limits	Only agent that touches the database, read-only. Never uses a quarantined result	Never writes SQL, never talks to the user. Same verdict every time	3 retries per question. Never retries unsafe or unreachable cases	Pauses only for personal data. Cannot change the query

What the Monitor checks, and why it has no model

A 3B model checking another 3B model shares its blind spots, so the Monitor uses rules worked out from the data: no negative money, nothing above all-time gross sales (Rs 12,99,209), no single order above the largest real order (Rs 16,189), no count above the largest table (757 rows), and no SUM or COUNT next to a plain column without GROUP BY. It also rejects an answer that is not backed by a valid result or misquotes the number.

5 Memory and RAG Unit 3



Checkpointer (SQLite, per `thread_id`): follow-ups work and a crashed run resumes from its last step.

Store (SQLite, namespace `(user_id, "glossary")`): a user's own words survive a restart.

Error guide (Chroma, 15 sections, nomic-embed-text): grounded advice, and "I don't know" when the guide is silent.

Failure log (SQLite): every failure, its SQL and what Recovery decided.

6 Component checklist

every row has a trace you can open

ID	What we built	Evidence
U3-1	SQLite checkpointer + `thread_id`. Without it "And in September?" has no context; with it the follow-up works (55, then 63 delivered orders)	`traces/ta2/amnesia.txt, restart.txt`
U3-2	SQLite store. "Premium product = price above Rs 1500" saved in one thread, used in a new process and thread	`traces/ta2/glossary.txt`
U3-3	Chroma over our own error guide. k=1 misses half of a two-part question, k=3 answers both; "connect to MySQL" gets "I don't know"	`traces/ta2/rag.txt, ta2/corpus/error_guide.md`
U3-4	TA1 tool kept. Failures routed, explained with guide advice, logged. Resume after a crash from the last checkpoint	`traces/ta2/advice.txt, resume.txt, faillog.txt`
U4-1	Five-field role table (section 4)	`ta2/ROLES.md`
U4-2	Executor, Monitor, Recovery pattern from our topic row. The retry budget is the hop limit	`traces/ta2/annotated/routing_silent_failure.md, budget.txt`
U4-3	Human approval before personal data is shown. Approve shows it, reject masks it	`traces/ta2/privacy.txt`
U4-4	Error propagation, measured over 10 runs per mode (section 9)	`traces/ta2/containment.txt, traces/eval/`
EV	Same 5 cases on TA1 and TA2, 3 runs each (section 8)	`traces/eval/results.json`

7 Five injected failure types and the retry budget

Failure type	How it is caught	Detected	Outcome (3 runs)	Avg retries
Schema: ambiguous `order_id`	tool error, guide advice	3/3	3/3 correct	2.0
Syntax: stray bracket in the query	tool error, guide advice	3/3	3/3 correct	1.0
Unsafe: "delete all cancelled orders"	tool refuses, no retry	3/3	3/3 safe refusal	0.0
Unavailable: database down	tool error, no retry	3/3	3/3 safe stop	0.0
Bad result: values 100x too big, no error	Monitor bounds, quarantine	3/3	3/3 correct	1.0

Lock clears in time

A lock that clears after two tries recovers inside the budget: 5 customers in Pune, correct.

Budget used up

A lock that never clears uses all 3 retries. The Monitor rejects the unsupported answer and Recovery stops, naming the cause. `traces/ta2/budget.txt`

8 PBA 1 vs PBA 2 on the same five cases

EV

Case	PBA 1 (single agent)	PBA 2 (team)	Model calls 1 / 2	Time (s) 1 / 2
Clean question	3/3 correct	3/3 correct	2.0 / 3.0	2.9 / 3.6
Needs a schema fix	2/3 correct, 1/3 wrong	3/3 correct	3.0 / 5.0	4.3 / 6.7
Unsafe request	3/3 safe	3/3 safe	2.0 / 1.0	0.9 / 0.4
Database down	3/3 safe	3/3 safe	2.0 / 1.0	0.8 / 0.6
Silent corruption (x100)	3/3 wrong	3/3 safe stop	2.0 / 6.0	1.6 / 5.2

qwen2.5:3b, temperature 0, RTX 4060. Judged by fixed rules against known answers, never by a model. Answers in `traces/eval/results.json`; rerun with `python ta2/evaluate.py`.

What got better

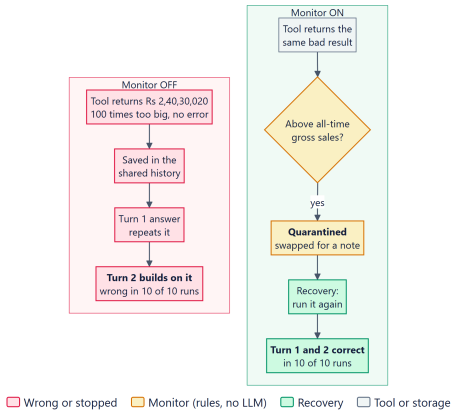
- Wrong numbers given to the user: 4 of 15 runs down to none.
- Silent corruption: PBA 1 reported Rs 24,03,00,20 every time; PBA 2 caught it and refused to guess.
- Unsafe and unreachable cases end after one model call, half of PBA 1's cost.

What got worse

- 2 to 3 times the calls and time when something needs fixing (corrupt: 6.0 vs 2.0).
- One extra call on the clean question: the team rules in the prompt make the model drop GROUP BY, and the Monitor sends it back.
- A fault that never clears uses all 3 retries before stopping.

9 Measured challenge: error propagation

U4-4, 10 two-turn runs per mode



One result is corrupted (x100) in turn 1. Turn 2 asks "How does that compare with August 2026?" in the same thread, so it reuses whatever turn 1 left in the shared history.

Monitor	Bad value in history	Carried to turn 2	Turn 1 correct
off	10/10	10/10	0/10
on	0/10	0/10	10/10

How it spreads: the bad number goes into the answer, the answer stays in the shared history, and the next turn builds on it. **How containment works:** the Monitor replaces the bad ToolMessage with a quarantine note that has the same message id, before any agent reads it.

Cost: 6.0 vs 4.0 model calls, 5.5 vs 3.5 seconds per question.

Found by our own evaluation

The first full run showed PBA 2 wrong on the clean question 3 times out of 3. The Executor had written `SELECT p.name, SUM(...)` with no GROUP BY. SQLite runs that without an error and returns one row: the whole month's revenue next to one arbitrary product. Our number bounds could not see it, because the total was real. The same bug gave PBA 1's wrong answer (Rs 8,13,186.80 for one order). We fixed the checks, not the judge: the Monitor now reads the SQL, Recovery names the GROUP BY to add, and the guide says never to hide bad rows with a filter. Clean question: 3/3 wrong became 3/3 correct.

10 Limits

- **Small samples.** 3 runs per case, 10 for containment. At temperature 0 they show stability more than spread.
- **The Monitor only knows what we wrote down.** A wrong answer inside the limits, such as filtering order lines instead of order totals, still gets through.
- **The model invents stand-ins** for data we do not have (loyalty points). Neither version catches it.
- **One model, one database.** A bigger model makes fewer SQL mistakes, so the cost gap would shrink.

11 Next steps

- Stop early when the same check fails twice with the same numbers.
- Let owners add their own limits next to their glossary ("an order is never above Rs 50,000").
- Real Postgres or MySQL through a read-only user, with the same checks.
- A page that shows the answer, the SQL behind it, and every time the Monitor stepped in.

12 Contribution

Member	Built	Can also explain
Swaraj Mandre ADT23SOCB1192	PBA 1 agent and its refinement; PBA 2 graph wiring, Monitor rules, retry budget, containment demo	Memory
Saumya Joshi ADT23SOCB0997	Memory: checkpointer and resume, SQLite store and glossary, failure log, privacy approval gate	RAG
Siddhant Pawar ADT23SOCB1117	RAG: error guide, Chroma index, Recovery agent; evaluation script and tables	PBA 1 agent